

# 增量 SVD 中一个开放问题的解答

张杨文

2022 年 5 月 3 日

## 摘要

增量奇异值分解由 Brand 提出, 用于高效计算矩阵的 SVD。该算法需要计算数千或数百万个正交矩阵并将它们相乘。然而, 乘法可能会破坏正交性。因此, 在实践中需要许多重正交化。在 [Linear Algebra and its Applications 415 (2006) 20-30] 中, Brand 问道: “为了保证一定的整体数值精度水平, 需要多久进行一次重正交化是一个开放问题; 这不会改变整体复杂度。” 在本文中, 我们回答了这个问题, 答案是我们可以避免计算大量这些正交矩阵, 因此通过修改他的算法, 重正交化并不是必需的。我们证明了这种修改不会改变算法的结果。数值实验说明了我们修改的性能。

## 1 引言

矩阵的奇异值分解在许多应用中都有应用, 例如本征正交分解模型降阶和主成分分析。

SVD 的一个缺点是计算成本高。设  $U$  是一个  $m \times n$  的低秩  $r$  稠密矩阵, 传统方法的计算复杂度为  $O(mn^2 + m^2n + n^3)$  时间, 这对于大规模矩阵是不可行的。Lanczos 方法 [2] 产生薄 SVD, 复杂度为  $O(mnr^2)$  时间, 但我们需要提前知道秩。这些方法被称为批处理方法, 因为它们需要存储矩阵。

在某些场景中, 数据集是增量产生的, 例如时间相关偏微分方程的快照。随着矩阵的列变得可用时执行 SVD 可能是有利的, 而不是等待所有数据集可用后才开始计算。数据集的这些可用性特性催生了一类增量方法。

2002 年, Brand [3] 提出了一种新算法来增量地计算矩阵的 SVD。给定矩阵  $U = Q\Sigma R^T$  的 SVD, 目标是通过使用  $U$  的 SVD 和新添加的向量  $c$  来更新相关矩阵  $[U \mid c]$  的 SVD。然后可以通过以下步骤构造  $[U \mid c]$  的 SVD:

1. 令  $e = c - QQ^\top c$  且  $p = \|e\|$
2. 找到  $\begin{bmatrix} \Sigma & Q^\top c \\ 0 & p \end{bmatrix} = \tilde{Q}\tilde{\Sigma}\tilde{R}^\top$  的完全 SVD, 然后
3. 通过下式更新  $[U \mid c]$  的 SVD

$$[U \mid c] = ([Q \mid e/p]\tilde{Q})\tilde{\Sigma}\left(\begin{bmatrix} R & 0 & 0 \\ 1 \end{bmatrix}\tilde{R}\right)^\top.$$

对于许多激励性应用, 只需要  $U$  的主要奇异向量和值。因此, 在实践中, 当  $p$  或  $\tilde{\Sigma}$  的最后一个奇异值很小时, 我们执行截断。增量 SVD 已应用于许多不同领域; 更多细节见 [8, 10-12]。还有许多其他增量方法, 见 [1] 的综述。

有两种更新左右奇异子空间的方法。第一种方法非常直接, 更新由下式给出

$$Q \leftarrow [Q \mid e/p]\tilde{Q}, \quad \Sigma \leftarrow \tilde{\Sigma}, \quad R \leftarrow \begin{bmatrix} R & 0 & 0 \\ 1 \end{bmatrix}\tilde{R}. \quad (1.1)$$

然而, 这种更新有两个缺陷:

- (1) (1.1) 的计算成本很高, 因为更新涉及非常高而瘦的矩阵  $Q$  和  $R$ 。
- (2) 几乎每次更新后都需要对  $Q$  进行重正交化。理论上, 矩阵  $[Q \mid e/p]\tilde{Q}$  是正交的。然而, 在实践中, 微小的数值误差会导致正交性损失。如果没有重正交化, 增量 SVD 算法的分解就不是正交的。因此, 奇异值不真实, 截断不可靠。因此需要重正交化, 如果矩阵的秩不是非常低, 计算成本可能会很高。具体来说, Fareed 等人在 [7] 中推荐使用 Gram-Schmidt 正交化, 而 Oxberry 等人在 [13] 中选择使用薄 QR 分解。

例 1 中的数值实验表明, 上述两项占用了 CPU 时间的大部分。

为了避免巨大的计算成本, Brand [4] 建议间接更新矩阵  $Q$  和  $R$ 。他的想法是将 SVD 分解为五个矩阵

$$U = Q_{\text{old}}\tilde{Q}_{\text{old}}\Sigma_{\text{old}}\tilde{R}_{\text{old}}^\top R_{\text{old}}^\top$$

其中  $Q_{\text{old}}\tilde{Q}_{\text{old}}$ 、 $R_{\text{old}}\tilde{R}_{\text{old}}$ 、 $Q_{\text{old}}$  和  $\tilde{Q}_{\text{old}}$  (但不是  $\tilde{R}_{\text{old}}$  或  $R_{\text{old}}^\top$ ) 是正交的。大的外部矩阵仅记录左右子空间的跨度, 并通过向  $Q_{\text{old}}$  追加列和向  $R_{\text{old}}$  追加行来构建。更新仅涉及小矩阵  $\tilde{Q}_{\text{old}}$  和  $\tilde{R}_{\text{old}}$ 。这使得更新更快。

然而, 经过数千或数百万次更新, 乘法可能会因数值误差而侵蚀正交性; 见图 3 中的右图。因此, 需要对这些小正交矩阵进行重正交化。在 [4]

中, Brand 问道: “为了保证一定的整体数值精度水平, 需要多久进行一次重正交化是一个开放问题; 这不会改变整体复杂度。” 我们还注意到, 对于某些数据集, 仅对小矩阵  $\tilde{Q}$  进行重正交化是不够的。在例 2 中, 我们表明, 如果不对外部大矩阵  $Q_{\text{old}}$  进行重正交化, 算法会提供不真实的 SVD。

在本文中, 我们首先回答这个开放问题, 答案是我们可以避免计算大量这些正交矩阵, 因此通过修改 Brand 的算法, 对一些小正交矩阵的重正交化不是必需的。其次, 我们为外部大矩阵  $Q_{\text{old}}$  提出了一种有效的重正交化方法, 例 2 和例 3 中的数值实验表明, 该重正交化方法在实践中效果良好。第三, 我们证明了我们的修改不会改变 Brand 算法的输出。

## 2 增量 SVD (I)

我们首先介绍本文所需的符号。令  $I_k$  为  $k \times k$  单位矩阵, 并定义  $W$ -加权内积:

$$(a, b)_W = a^\top W b, \quad a, b \in \mathbb{R}^m.$$

设  $U$  为  $m \times n$  稠密矩阵,  $u_j$  表示  $U$  的第  $j$  列, 即

$$U = [u_1 \mid u_2 \mid \cdots \mid u_n].$$

为方便起见, 本文采用 Matlab 符号。我们使用  $U(:, 1:\ell)$  和  $U_\ell$  表示  $U$  的前  $\ell$  列, 使用  $U(1:\ell, 1:\ell)$  表示  $U$  的  $\ell$  阶首主子式。函数  $\text{diag}: \mathbb{R}^k \rightarrow \mathbb{R}^{k \times k}$  输入一个向量, 输出一个对角矩阵, 该向量的条目在其主对角线上, 如下所示: 如果  $\alpha = (a_1, a_2, \dots, a_k)^\top$ , 则

如果数据来自 PDE 的 Galerkin 型模拟, 直接应用 Brand 算法时, 需要处理加权矩阵的 Cholesky 分解。为避免这种情况, Fareed 等人 [7] 扩展了 Brand 算法以处理此类数据, 而无需计算 Cholesky 分解。更具体地说, 如果数据位于有限元空间中, 并使用一组基函数表示。那么数据的 SVD 等价于找到其系数矩阵的所谓核心 SVD。

定义 1. [7] 矩阵  $U \in \mathbb{R}^{m \times n}$  的核心 SVD 是一种分解  $U = Q\Sigma R^\top$ , 其中  $Q \in \mathbb{R}^{m \times d}$ ,  $\Sigma \in \mathbb{R}^{d \times d}$ , 且  $R \in \mathbb{R}^{n \times d}$  满足

$$Q^\top W Q = I, \quad R^\top R = I, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_d),$$

其中  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_d > 0$ 。值  $\{\sigma_i\}$  称为  $U$  的（正）奇异值， $Q$  和  $R$  的列称为  $U$  的相应奇异向量。

我们注意到，如果  $W = I$ ，则上述核心 SVD 简化为矩阵  $U$  的标准 SVD。在 [7] 中， $W$  是标准有限元质量矩阵或刚度矩阵。关于加权范数设置下增量 SVD 的更多信息，请参见 [5,6]。

接下来，我们介绍 Brand 的增量 SVD 算法，并遵循 [7] 中的讨论。

步骤 1：初始化。假设  $U$  的第一列非零，即  $u_1 \neq 0$ ，我们通过设置下式初始化  $u_1$  的核心 SVD

$$\Sigma = (u_1^\top W u_1)^{1/2}, \quad Q = u_1 \Sigma^{-1}, \quad R = 1.$$

该算法如算法 1 所示。

步骤 2： $U_\ell$  的核心 SVD。假设我们已经有了  $U_\ell$  的秩为  $k$  的截断核心 SVD：

$$U_\ell = Q \Sigma R^\top, \quad \text{with} \quad Q^\top W Q = I_k, \quad R^\top R = I_k, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_k), \quad (2.1)$$

其中  $\Sigma \in \mathbb{R}^{k \times k}$  是对角矩阵，对角线上是  $U_\ell$  的  $k$  个（有序）奇异值， $Q \in \mathbb{R}^{m \times k}$  是包含  $U_\ell$  对应  $k$  个左奇异向量的矩阵， $R \in \mathbb{R}^{\ell \times k}$  是包含  $U_\ell$  对应  $k$  个右奇异向量的矩阵。

算法 1（初始化增量 SVD）

输入： $u_1 \in \mathbb{R}^m$ ,  $W \in \mathbb{R}^{m \times m}$

1:  $\Sigma = (u_1^\top W u_1)^{1/2}$ ;  $Q = u_1 \Sigma^{-1}$ ;  $R = 1$ ;

2: return  $Q, \Sigma, R$

步骤 3：更新  $U_{\ell+1}$  的核心 SVD。我们下一步的目标是更新上述 SVD，即我们希望通过使用  $U_\ell$  的核心 SVD 和新添加的向量  $u_{\ell+1}$  来找到  $U_{\ell+1}$  的核心 SVD。

为此，令  $e \in \mathbb{R}^m$  为  $u_{\ell+1}$  在由  $Q$  的列张成的空间上关于加权内积  $(\cdot, \cdot)_W$  的残差。因此，

$$e = u_{\ell+1} - Q Q^\top W u_{\ell+1}.$$

令  $p$  为  $e$  的模，即  $p = (e^\top W e)^{1/2}$ 。如果  $p > 0$ ，我们令  $\tilde{e}$  为  $e$  方向上的单位向量，即  $\tilde{e} = e/p$ 。如果  $p = 0$ ，我们设  $\tilde{e} = 0$ 。那么我们有基本恒等式：

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = [Q \Sigma R^\top \mid u_{\ell+1}] = [Q \mid \tilde{e}] \left[ \underbrace{\begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \\ 0 & p \end{bmatrix}}_p \right] \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (2.2)$$

我们可以通过找到上述恒等式右侧矩阵  $Y$  的完全 SVD 来找到更新后矩阵  $U_{\ell+1}$  的 SVD。

(1) 如果  $p < \text{tol}$ , 我们近似并设  $p = 0$  和  $\tilde{e} = 0$ 。令  $Q_Y \Sigma_Y R_Y^\top$  为  $Y = \begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \\ 0 & 0 \end{bmatrix}$  的完全 SVD。那么  $U_{\ell+1} \in \mathbb{R}^{m \times (\ell+1)}$  的核心 SVD 由下式给出

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = ([Q \mid 0] Q_Y) \Sigma_Y \left( \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y \right)^\top.$$

这表明了以下更新

$$Q \leftarrow Q Q_Y(1:k, 1:k), \quad \Sigma \leftarrow \Sigma_Y(1:k, 1:k), \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(\cdot, 1:k). \quad (2.3)$$

(2) 如果  $p \geq \text{tol}$ , 我们令  $Q_Y \Sigma_Y R_Y^\top$  为 (2.2) 中中间矩阵  $Y$  的完全 SVD, 然后通过下式更新  $U_{\ell+1}$  的 SVD

$$Q \leftarrow [Q \mid \tilde{e}] Q_Y \quad \Sigma \leftarrow \Sigma_Y \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y. \quad (2.4)$$

在 [7] 中, 作者考虑了来自二维圆柱绕流模拟的数据。增量 SVD 算法持续计算非常小的奇异值 (接近机器精度) ——这需要巨大的计算成本, 并且这些值在它们的应用中并不需要。因此, 他们在 [7, 算法 4] 中建议, 如果最后几个奇异值小于容差, 则进行另一种截断。即, 找到最小的  $r$  使得  $\Sigma_Y(r, r) \geq \text{tol}$  且  $\Sigma_Y(r+1, r+1) < \text{tol}$  (原文此处可能有笔误, 应为  $\Sigma_Y(r+1, r+1) < \text{tol}$  以确定截断位置), 并设置以下截断:

$$Q \leftarrow Q(\cdot, 1:r), \quad \Sigma \leftarrow \Sigma(1:r, 1:r), \quad R \leftarrow R(\cdot, 1:r), \quad (2.5)$$

其中  $Q, \Sigma$  和  $R$  在 (2.4) 中定义。现在我们将上述讨论总结在算法 2 中。

算法 2 (更新增量 SVD (I))

输入:  $Q \in \mathbb{R}^{m \times k}$ ,  $\Sigma \in \mathbb{R}^{k \times k}$ ,  $R \in \mathbb{R}^{k \times k}$ ,  $u_{+1} \in \mathbb{R}^m$ ,  $W \in \mathbb{R}^{m \times m}$ ,  $\text{tol}$

```

1: 设  $d = Q^T (W u_{+1})$ ;  $e = u_{+1} - Q d$ ;  $p = (e^T W e)^{1/2}$ ;
2: if  $p < \text{tol}$  then
3:   设  $p = 0$ ;
4: else
5:   设  $e = e / p$ ;
6: end if
7: 设  $Y = [\Sigma, d; 0, p]$ ;
8:  $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$ ;
9: if  $p < \text{tol}$  then
10:  设  $Q = Q Q_Y(1:k, 1:k)$ ,  $\Sigma = \Sigma_Y(1:k, 1:k)$ ,  $R = [R, 0; 0, 1] R_Y(:, 1:k)$ ;
11: else
12:  设  $Q = [Q, e] Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R = [R, 0; 0, 1] R_Y$ .
13:  if  $\Sigma(r, r) \geq \text{tol} \ \&\& \ \Sigma(r+1, r+1) < \text{tol}$  then % 根据描述调整截断条件
14:    设  $Q = Q(:, 1:r)$ ,  $\Sigma = \Sigma(1:r, 1:r)$ ,  $R = R(:, 1:r)$ ;
15:  end if
16: end if
17: return  $Q, \Sigma, R$ 

```

接下来, 我们讨论 Brandt 算法 (算法 2) 在  $W = I$  时的复杂度, 并注意  $U \in \mathbb{R}^{m \times n}$  具有低秩  $r$ 。总时间复杂度为  $\mathcal{O}((m+n)r^2)$ , 细节如下所示。

1. 第 1 行: 需要执行  $n$  次, 每次的计算成本约为  $\mathcal{O}(mr)$ , 总复杂度为  $\mathcal{O}(mnr)$ 。2. 第 8 行: 由于  $Y$  是一个单列边界的对角矩阵, 因此可以在  $\mathcal{O}(r^2)$  时间内进行双对角化, 总时间为  $\mathcal{O}(nr^2)$ 。3. 第 10-12 行: 每次的计算成本约为  $\mathcal{O}(mr^2 + lr^2)$ , 因此总复杂度约为  $\mathcal{O}(\sum_{\ell=1}^n (mr^2 + lr^2)) = \mathcal{O}((m+n)nr^2)$ 。

步骤 4: 重正交化。理论上, 上述 SVD 更新产生正交的左右奇异向量。然而, 在实践中, 微小的数值误差会导致正交性损失。那么增量 SVD 算法就是一个非正交分解, 奇异值不真实, 因此截断不可靠。因此需要重正交化, 如果矩阵的秩不是非常低, 计算成本可能很高。具体来说, Brand 推荐使用 Gram-Schmidt 正交化 [3], 而 Oxberry 等人在 [13] 中选择使用薄 QR 分解。我们的数值测试表明, 当加权矩阵是单位矩阵时, 薄 QR 分解比 Gram-Schmidt 更快, 参见例 1。但是, 由于目前没有适用于加权范数设置的薄 QR 分解, Fareed 等人在 [7] 中使用  $W$ -加权 Gram-Schmidt 进行重

正交化。

算法 3 (重正交化)

输入:  $Q \in \mathbb{R}^{m \times k}$ ,  $W \in \mathbb{R}^{m \times m}$ , tol

```

1: if  $|(Q(:,end))^T W Q(:,1)| > \text{tol}$  then
2:   for  $i = 1$  to  $k$  do
3:     设  $q = Q(:,i)$ 
4:     for  $j = 1$  to  $i-1$  do
5:        $Q(:,i) = Q(:,i) - (q^T W Q(:,j)) Q(:,j)$ 
6:     end for
7:     设  $\text{norm} = ((Q(:,i))^T W Q(:,i))^{1/2}$ 
8:     设  $Q(:,i) = Q(:,i) / \text{norm}$ 
9:   end for
10: end if
11: return  $Q$ 

```

我们不讨论重正交化的时间复杂度，因为不清楚需要执行算法 3 多少次。然而，例 1 中的数值实验表明，上述正交化步骤是增量 SVD 算法计算成本的一大部分。

最后，我们在算法 4 中总结了带有  $W$ -加权 Gram-Schmidt 重正交化的增量 SVD 的完整实现。

算法 4 (完全增量 SVD (I))

输入:  $W \in \mathbb{R}^{m \times m}$ , tol

```

1: 获取  $u_1$ 
2:  $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$  % 算法 1
3: for  $i = 2, \dots, n$  do
4:   获取  $u_i$ 
5:    $[Q, \Sigma, R] = \text{UpdateISVD1}(Q, \Sigma, R, u_i, W, \text{tol})$  % 算法 2
6:    $Q = \text{Reorthogonalization}(Q, W, \text{tol})$  % 算法 3
7: end for
8: return  $Q, \Sigma, R$ 

```

例 1. 在此示例中，我们测试算法 4。令  $\Omega \subset \mathbb{R}^2$  为单位正方形，我们将其划分为 524288 个均匀三角形。令  $\{\phi_i\}_{i=1}^m$  为标准线性有限元基函数。接

下来, 令  $\{(x_i, y_j)\}_{i,j=1}^m$  为  $\Omega$  的网格点,  $\{t_i\}_{i=1}^n$  为  $[0, 10]$  上的等距网格, 时间步长  $\Delta t = 1/10^3$ 。定义

$$f(t, x, y) = \cos(t(x + y)), \quad b_i = [(f(t_i), \phi_j)]_{j=1}^m, \quad B = [b_1 | b_2 | \dots | b_n],$$

$$M = [(\phi_j, \phi_i)]_{i,j=1}^m, \quad u_k = [f(t_k, x_i, y_j)]_{i,j=1}^m, \quad U = [u_1 | u_2 | \dots | u_n].$$

这里  $M$  是有限元质量矩阵,  $u_k$  是  $f(t, x, y)$  在  $t = t_k$  时的线性拉格朗日插值的系数。我们通过设置  $\text{tol} = 10^{-12}$  计算矩阵  $B$  在  $W = I$  下的 SVD 和矩阵  $U$  在  $W = M$  下的 SVD。令  $\|A\|_F$  为矩阵  $A$  的 Frobenius 范数, 我们使用

$$\mathcal{E}_W = \|I - Q^\top W Q\|$$

来度量正交矩阵  $Q$  在加权内积  $(\cdot, \cdot)_W$  下的正交性误差。

在图 1 中, 我们展示了左奇异矩阵  $Q$  在不同加权范数下的正交性。对于  $W = M$  的情况,  $Q$  的正交性被侵蚀 (即使进行了重正交化)。

图 1: 例 1: 左:  $W = I$ 。右:  $W = M$ 。

整个模拟对于  $W = I$  和  $W = M$  分别花费约 154 秒和 4000 秒。接下来, 我们在表 1 中展示了算法 4 主要部分的 CPU 时间 (秒)。我们看到, 正交化步骤是增量 SVD 算法计算成本的一大部分, 尤其是在  $W = M$  的情况下。

表 1: 不同加权内积下的 CPU 时间 (秒)。

|         | 算法 2 第 1 行 | 算法 2 第 9-13 行 | 算法 3   |
|---------|------------|---------------|--------|
| $W = I$ | 32.03      | 460.21        | 360.13 |
| $W = M$ | 110.58     | 115.96        | 3761.6 |

### 3 增量 SVD (II)

算法 4 易于实现, 但计算成本高, 特别是在加权范数设置下。对于一个  $m \times n$  的低秩  $r$  稠密矩阵, 算法 4 的时间复杂度为  $\mathcal{O}(m(m+n)r^2)$ 。在 [4] 中, Brand 将复杂度降低到  $\mathcal{O}(mnr)$ 。然而, 为了更新左奇异空间, 必须计算数千或数百万个正交矩阵并将它们相乘。更新右奇异空间更为复杂, 需要计算数千或数百万次矩阵的伪逆。



接下来, 我们遵循 [4] 将 Brand 的改进扩展到计算关于加权内积  $(\cdot, \cdot)_W$  的矩阵  $U \in \mathbb{R}^{m \times n}$  的核心 SVD。我们不按照 (2.3) 或 (2.4) 中规定的那样更新大的奇异向量矩阵, 而是假设  $U_\ell \in \mathbb{R}^{m \times \ell}$  的核心 SVD 可以写成以下五个矩阵

$$U_\ell = Q_{\text{old}} \tilde{Q}_{\text{old}} \Sigma_{\text{old}} \tilde{R}_{\text{old}}^\top R_{\text{old}}^\top. \quad (3.1)$$

这里  $Q_{\text{old}} \in \mathbb{R}^{m \times k}$  是在加权内积下正交的矩阵,  $\tilde{Q}_{\text{old}}$  和  $\tilde{R}_{\text{old}}^\top R_{\text{old}}^\top$  是标准正交矩阵, 但  $\tilde{R}_{\text{old}}^\top$  和  $R_{\text{old}}^\top$  不是。大的外部矩阵仅记录左右子空间的跨度, 并通过向  $Q_{\text{old}}$  追加列和向  $R_{\text{old}}^\top$  追加行来构建。并且更新仅涉及小矩阵  $\tilde{Q}_{\text{old}}$  和  $\tilde{R}_{\text{old}}^\top$ , 这使得更新更快。

接下来, 我们考虑  $U_{\ell+1} = [U_\ell \mid u_{\ell+1}] \in \mathbb{R}^{m \times (\ell+1)}$  的核心 SVD。类似于第 2 节, 令  $e \in \mathbb{R}^m$  为  $u_{\ell+1}$  在由  $Q_{\text{old}} \tilde{Q}_{\text{old}}$  的列张成的空间上关于加权内积  $(\cdot, \cdot)_W$  的残差, 即

$$e = u_{\ell+1} - (Q_{\text{old}} \tilde{Q}_{\text{old}})(Q_{\text{old}} \tilde{Q}_{\text{old}})^\top W u_{\ell+1} = u_{\ell+1} - Q_{\text{old}} Q_{\text{old}}^\top W u_{\ell+1}.$$

令  $p = (e^\top W e)^{1/2}$ 。如果  $p > 0$ , 我们令  $\tilde{e} = e/p$ , 否则设  $\tilde{e} = 0$ 。然后由 (3.1) 和基本恒等式 (2.2) 我们有

$$U_{\ell+1} = \left[ Q_{\text{old}} \tilde{Q}_{\text{old}} \mid \tilde{e} \right] \underbrace{\begin{bmatrix} \Sigma_{\text{old}} & \tilde{Q}_{\text{old}} Q_{\text{old}}^\top W u_{\ell+1} \\ 0 & p \end{bmatrix}}_Y \begin{bmatrix} R_{\text{old}} \tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix}^\top.$$

假设  $Q_Y \Sigma_Y R_Y^\top$  是  $Y$  的完全 SVD, 那么  $U_{\ell+1}$  的核心 SVD 由下式给出

$$U_{\ell+1} = \left( \left[ Q_{\text{old}} \tilde{Q}_{\text{old}} \mid \tilde{e} \right] Q_Y \right) \Sigma_Y \left( \begin{bmatrix} R_{\text{old}} \tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} R_Y \right)^\top. \quad (3.2)$$

(1) 如果  $p < \text{tol}$ , 我们设  $p = 0$  和  $\tilde{e} = 0$ , 则更新不会增加秩。我们通过下式更新小正交矩阵  $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times k}$  和对角矩阵  $\Sigma_{\text{new}} \in \mathbb{R}^{k \times k}$

$$\tilde{Q}_{\text{new}} \longleftarrow \tilde{Q}_{\text{old}} Q_Y(1:k, 1:k), \quad \Sigma_{\text{new}} \longleftarrow \Sigma_Y(1:k, 1:k). \quad (3.3)$$

(2) 如果  $p \geq \text{tol}$ , 我们然后通过将向量  $\tilde{e}$  添加到  $Q_{\text{old}}$  来更新大的外部矩阵  $Q_{\text{new}}$ , 并通过下式更新小正交矩阵  $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times (k+1)}$  和对角矩阵  $\Sigma_{\text{new}} \in \mathbb{R}^{(k+1) \times (k+1)}$

$$Q_{\text{new}} \leftarrow [Q_{\text{old}} \mid \tilde{e}], \quad \tilde{Q}_{\text{new}} \leftarrow \begin{bmatrix} \tilde{Q}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} Q_Y, \quad \Sigma_{\text{new}} \leftarrow \Sigma_Y. \quad (3.4)$$

(3.3) 或 (3.4) 的时间复杂度约为  $\mathcal{O}(r^2)$ , 因为左奇异向量矩阵的更新是通过两个小正交矩阵的乘法完成的。这与 (2.3) 或 (2.4) 相比是一个主要区别。

然而, 我们必须在数千或数百万次更新中更新小正交矩阵, 乘法可能会因数值误差而侵蚀  $\tilde{Q}$  的正交性。因此, 当  $\tilde{Q}$  的第一个和最后一个左奇异向量之间的内积大于某个容差时, 需要进行重正交化。

在 [4] 中, Brand 问道: “为了保证一定的整体数值精度水平, 需要多久进行一次重正交化是一个开放问题; 这不会改变整体复杂度”。在下一节中, 我们回答这个问题, 答案是我们 \*\* 可以避免 \*\* 计算大量那些正交矩阵, 并且这些乘法不是必需的。因此, 不需要重正交化。

1. 理论上, 新添加的向量  $\tilde{e}$  与  $Q_{\text{old}}$  的每一列正交。然而, 在实践中, 对于某些数据集, 矩阵  $[Q_{\text{old}} \mid \tilde{e}]$  非常快地失去正交性。因此需要对矩阵进行重正交化。我们注意到, 在 [4] 中, Brand 没有包括这一点。在例 2 中, 我们展示了这有多重要, 并在下一节中提出了一种非常有效的方法来解决这个问题。

右奇异向量矩阵  $R_{\text{new}}$  和  $\tilde{R}_{\text{new}}$  的更新更为复杂, 因为需要在保证乘积  $R_{\text{new}} \tilde{R}_{\text{new}}$  的列是正交的同时向  $R_{\text{old}}$  添加行。从基本恒等式 (2.2) 和 (3.2) 可知, 右侧更新必须满足

在以下更新中, 我们需要计算  $\tilde{R}_{\text{old}}$  的伪逆和  $R_Y$  的子矩阵。我们用  $X^+$  表示  $X$  的伪逆。

- (3) 如果  $p < \text{tol}$ , 秩不增加,  $R_Y$  的最后一列表示一个未使用的子空间维度, 应被抑制。令  $\mathcal{R}_1 = R_Y(1:k, 1:k)$ ,  $\mathcal{R}_2 = R_Y(k+1, 1:k)$ ,

$$\tilde{R}_{\text{new}} \leftarrow \tilde{R}_{\text{old}} \mathcal{R}_1, \quad \tilde{R}_{\text{new}}^+ \leftarrow \mathcal{R}_1^+ \tilde{R}_{\text{old}}^+, \quad R_{\text{new}} \leftarrow \begin{bmatrix} R_{\text{old}} & \mathcal{R}_2 \tilde{R}_{\text{new}}^+ \end{bmatrix}. \quad (3.5)$$

- (4) 如果  $p \geq \text{tol}$ , 更新是秩增加的。我们通过下式更新小正交矩阵  $\tilde{Q}_{\text{new}} \in \mathbb{R}^{m \times k}$  和对角矩阵  $\Sigma_{\text{new}} \in \mathbb{R}^{k \times k}$

$$\tilde{R}_{\text{new}} \leftarrow \begin{bmatrix} \tilde{R}_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix} R_Y, \quad \tilde{R}_{\text{new}}^+ \leftarrow R_Y^+ \begin{bmatrix} \tilde{R}_{\text{old}}^+ & 0 \\ 0 & 1 \end{bmatrix}, \quad R_{\text{new}} \leftarrow \begin{bmatrix} R_{\text{old}} & 0 \\ 0 & 1 \end{bmatrix}. \quad (3.6)$$

该增量 SVD 算法的实现如算法 5 和 6 所示。

图 2:  $B$  的奇异值: 左: 算法 6。右: MATLAB 内置函数 `svd()`。

接下来, 我们向算法 6 添加重正交化 (细节将在下一节给出), 并绘制修改后的算法 6 与 MATLAB 内置函数 `svd()` 之间的误差 (图 3 左)。我们看到新添加的重正交化在实践中效果良好。

最后, 我们测试每次更新时小正交矩阵  $\tilde{Q}$  的正交性, 我们使用  $\mathcal{E}_I(\tilde{Q}) = \|I - \tilde{Q}^\top \tilde{Q}\|$  来度量  $\tilde{Q}$  的正交性, 如图 3 右所示, 我们看到即使在执行重正交化之后, 矩阵  $\tilde{Q}$  在  $n = 600$  后也很快失去正交性。

图 3: 例 2: 左: 通过添加重正交化后,  $B$  的前 34 个奇异值的误差。右: 每次更新时  $\tilde{Q}$  的正交性误差。

## 4 我们的改进

正如第 3 节所指出的, 算法 6 需要在数千或数百万次更新中更新小正交矩阵  $\tilde{Q}$ , 乘法可能会因数值误差而侵蚀  $\tilde{Q}$  的正交性。为了克服这个问题, Brand 提出当  $\tilde{Q}$  的第一个和最后一个左奇异向量之间的内积大于某个容差时进行重正交化。然而, 正如我们在例 2 中看到的, 即使我们执行了重正交化, 正交矩阵  $\tilde{Q}$  仍然可能失去正交性。此外, 为了更新右奇异空间, 算法 6 需要计算数千或数百万次矩阵的伪逆。

首先, 我们总结一下本节的贡献。

(1) 我们回答了 Brand 在 [4] 中提出的问题: “为了保证一定的整体数值精度水平, 需要多久进行一次重正交化是一个开放问题; 这不会改变整体复杂度”。(2) 我们证明了新算法的输出与 Brand 的算法相同。(3) 我们表明, 不需要计算数千或数百万次矩阵的伪逆来更新右奇异子空间。(4) 我们为矩阵  $[Q_{\text{old}} \mid \tilde{e}]$  提出了一种重正交化方法, 我们注意到这一步在 [4] 的原始算法中是没有的。在例 2 中我们已经证明了重正交化是必要的。

首先, 我们引入以下引理:

引理 1. 设  $A \in \mathbb{R}^{k \times k}$ ,  $\alpha \in \mathbb{R}^k$ , 并令  $Q_1 \Sigma_1 R_1^\top$  为  $[A \mid \alpha]$  的薄 SVD, 那么

$$\begin{bmatrix} Q_1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} R_1^\top \\ r_{k+1}^\top \end{bmatrix} \quad (4.1)$$

是  $\begin{bmatrix} A & \alpha \\ 0 & 0 \end{bmatrix}$  的一个 SVD, 其中  $r_{k+1} \in \mathbb{R}^{k+1}$  是一个单位向量, 并且与  $R_1$  的每一列正交。

接下来, 我们使用引理 1 来修改秩不增加时左奇异子空间的更新。假设  $Q\Sigma R^\top$  是  $U_\ell$  的秩为  $k$  的截断核心 SVD,  $u_{\ell+1} - QQ^\top W u_{\ell+1}$  的范数非常小, 我们将其近似为 0。因此,

$$U_{\ell+1} = [U_\ell \mid u_{\ell+1}] = Q [\Sigma \mid Q^\top W u_{\ell+1}] \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (4.2)$$

令  $Q_1 \Sigma_1 R_1^\top$  为  $[\Sigma \mid Q^\top W u_{\ell+1}]$  的薄 SVD, 那么  $U_{\ell+1}$  的核心 SVD 由下式给出

$$U_{\ell+1} = QQ_1 \Sigma_1 R_1^\top \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}^\top. \quad (4.3)$$

因此, 以下更新

$$Q \leftarrow QQ_1, \quad \Sigma = \Sigma_1, \quad R = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_1, \quad (4.4)$$

由于引理 1, 与 (2.3) 完全相同。

接下来, 我们考虑一个更一般的情况。假设矩阵  $C$  有  $s$  列, 并且  $C - QQ^\top WC$  的范数为零或非常小, 那么我们将其近似为 0, 然后有以下基本恒等式:

$$U_{\ell+1} = [U_\ell \mid C] = Q [\Sigma \mid Q^\top WC] \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix}^\top. \quad (4.5)$$

在本文中, 我们使用 MATLAB 内置函数 `svd(Y, 'econ')` 来找到  $Y$  的 SVD, 因为矩阵  $Y$  总是短而胖的。

#### 4.1 更新左右奇异空间

由于我们假设矩阵  $U$  是低秩的,那么可以合理地预期  $\{u_{\ell+1}, u_{\ell+2}, \dots, u_n\}$  中的大多数向量与  $Q \in \mathbb{R}^{m \times k}$  中的向量线性相关或几乎线性相关。不失一般性,我们假设接下来的两个向量  $u_{\ell+1}$  和  $u_{\ell+2}$ , 当将它们投影到由  $Q$  的列张成的空间上时, 它们的残差小于算法 4 中的  $\text{tol}$ 。换句话说, 我们假设

$$\|u_{\ell+1} - QQ^\top W u_{\ell+1}\|_W < \text{tol}, \quad \|u_{\ell+2} - QQ^\top W u_{\ell+2}\|_W < \text{tol}. \quad (4.6)$$

然后我们按照 Brand 的算法增量地找到  $U_{\ell+2}$  的 SVD:

(1) 找到  $[\Sigma \mid Q^\top W u_{\ell+1}]$  的 SVD, 并假设

$$[\Sigma \mid Q^\top W u_{\ell+1}] = Q_{(1)} \Sigma_{(1)} R_{(1)}^\top, \quad (4.7)$$

$$Q_{(1)} \in \mathbb{R}^{k \times k}, \Sigma_{(1)} \in \mathbb{R}^{k \times k}, R_{(1)} \in \mathbb{R}^{(k+1) \times k} \quad \text{且} \quad Q_{(1)}^\top Q_{(1)} = Q_{(1)} Q_{(1)}^\top = I_k.$$

(2) 通过下式更新  $U_{\ell+1}$  的 SVD

$$Q_{\ell+1} = QQ_{(1)}, \quad \Sigma_{\ell+1} = \Sigma_{(1)}, \quad R_{\ell+1} = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_{(1)}. \quad (4.8)$$

根据以上讨论, 更新 (4.8) 与 (2.3) 完全相同。接下来, 根据算法 4, 我们应该检查当  $u_{\ell+2}$  投影到由  $Q_{\ell+1}$  的列张成的空间上时, 其残差的大小。为此, 我们引入以下引理。

引理 2. 令  $Q \in \mathbb{R}^{k \times k}$  为一个正交矩阵, 则对于任意  $x \in \mathbb{R}^m$  和  $Q \in \mathbb{R}^{m \times k}$ , 我们有

$$QQ^\top x = (QQ)(QQ)^\top x.$$

由于  $Q_{(1)}$  是一个正交矩阵, 那么根据引理 2 和 (4.6), 我们有

$$\|u_{\ell+2} - Q_{\ell+1} Q_{\ell+1}^\top W u_{\ell+2}\|_W = \|u_{\ell+2} - QQ^\top W u_{\ell+2}\|_W < \text{tol}.$$

接下来, 我们按照与上述步骤 (1) 和 (2) 相同的论点来处理  $u_{\ell+2}$ 。

(3) 找到  $[\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2}]$  的 SVD, 并假设

$$[\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2}] = Q_{(2)} \Sigma_{(2)} R_{(2)}^\top, \quad (4.9)$$

$$Q_{(2)} \in \mathbb{R}^{k \times k}, \Sigma_{(2)} \in \mathbb{R}^{k \times k}, R_{(2)} \in \mathbb{R}^{(k+1) \times k} \quad \text{且} \quad Q_{(2)}^\top Q_{(2)} = Q_{(2)} Q_{(2)}^\top = I_k.$$

根据 4.8, 我们通过下式更新  $U_{\ell+2}$  的 SVD

$$\begin{aligned} Q_{\ell+2} &= Q_{\ell+1} Q_{(2)} = Q Q_{(1)} Q_{(2)}, \\ \Sigma_{\ell+2} &= \Sigma_{(2)}, \\ R_{\ell+2} &= \begin{bmatrix} R_{\ell+1} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)} = \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix} \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}. \end{aligned} \quad (4.10)$$

算法 4 通过首先计算  $Q_{\ell+1} = Q Q_{(1)}$ , 然后计算  $Q_{\ell+2} = Q_{\ell+1} Q_{(2)}$  来更新左奇异矩阵  $Q_{\ell+2}$ 。这种更新的缺点是计算成本高。为了避免这种情况, Brand 提出首先计算  $\tilde{Q} = Q_{(1)} Q_{(2)}$ , 然后通过计算  $Q \tilde{Q}$  来更新矩阵  $Q$ ; 详见算法 6。对于右奇异空间, 算法 4 首先计算  $R_{\ell+1} = \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_{(1)}$ , 然后计算  $R_{\ell+2} = \begin{bmatrix} R_{\ell+1} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$  来更新  $R_{\ell+2}$ 。为了节省 CPU 时间, 可以使用 Brand 的算法来更新右奇异子空间, 但必须计算  $R_{(1)}$  和  $R_{(2)}$  的子矩阵的伪逆。

从例 1 和例 2 中, 我们已经看到两种算法在左奇异向量矩阵的正交性损失方面都受到了影响。

下面的新算法可以在不改变算法 6 的复杂度的情况下避免上述问题。为了描述我们的新算法, 我们需要以下基本恒等式。

$$\begin{bmatrix} A & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} B & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} A & 0 \\ 0 & I_2 \end{bmatrix} \begin{bmatrix} B & 0 \\ 0 & 1 \end{bmatrix}, \quad A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{(n+1) \times n}. \quad (4.11)$$

根据基本恒等式 (4.11), 我们可以将  $R_{\ell+2}$  重写为

$$R_{\ell+2} = \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix} \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}.$$

由 (4.10) 可知,  $U_{\ell+2}$  的 SVD 由下式给出

$$U_{\ell+2} = QQ_{(1)}Q_{(2)}\Sigma_{(2)}R_{(2)}^\top \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix}^\top. \quad (4.12)$$

受 (4.12) 的启发, 如果我们能直接计算  $Q_{(1)}Q_{(2)}$  和  $\begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$ , 那么  $U_{\ell+2}$  的 SVD 更新可能会避免许多矩阵-矩阵乘法操作。

定理 1. 令  $\tilde{Q} = Q_{(1)}Q_{(2)}$ ,  $\tilde{\Sigma} = \Sigma_{(2)}$  和  $\tilde{R} = \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix} R_{(2)}$ , 则  $\tilde{Q}\tilde{\Sigma}\tilde{R}^\top$  是

$$[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}]$$

的一个薄 SVD。

证明。首先, 使用 (4.7) 我们有

$$[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}] = [Q_{(1)}\Sigma_{(1)}R_{(1)}^\top \mid Q^\top W u_{\ell+2}].$$

注意  $Q_{(1)}^\top Q_{(1)} = Q_{(1)}Q_{(1)}^\top = I_k$ , 并且根据基本恒等式 (4.5) 我们有

$$\begin{aligned} [\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}] &= Q_{(1)} [\Sigma_{(1)} \mid Q_{(1)}^\top Q^\top W u_{\ell+2}] \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top \\ &= Q_{(1)} [\Sigma_{\ell+1} \mid Q_{\ell+1}^\top W u_{\ell+2}] \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top, \end{aligned}$$

其中我们使用了 (4.8) 中的前两个恒等式。接下来我们使用 (4.9) 得到:

$$\begin{aligned} [\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}] &= Q_{(1)}Q_{(2)}\Sigma_{(2)}R_{(2)}^\top \begin{bmatrix} R_{(1)} & 0 \\ 0 & 1 \end{bmatrix}^\top \\ &= \tilde{Q}\tilde{\Sigma}\tilde{R}^\top. \quad \blacksquare \end{aligned}$$

定理 1 表明, 我们可以通过首先计算  $[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2}] \tilde{Q}\tilde{\Sigma}\tilde{R}^\top$  的薄 SVD, 然后通过下式更新它们来更新  $U_{\ell+2}$  的 SVD

$$Q_{\ell+2} = Q\tilde{Q}, \quad \Sigma_{\ell+2} = \tilde{\Sigma}, \quad R_{\ell+2} = \begin{bmatrix} R & 0 \\ 0 & I_2 \end{bmatrix} \tilde{R}. \quad (4.13)$$

现在我们将上述讨论扩展到更一般的情况。

定理 2. 令  $Q\Sigma R^\top$  为  $U_\ell$  的秩为  $k$  的截断核心 SVD,  $u_{\ell+j}$  为  $U$  的第  $(\ell+j)$  列,  $j = 1, 2, \dots, s$ 。如果当将它们投影到由  $Q$  的列张成的空间上时, 它们的残差小于算法 4 中的  $\text{tol}$ 。令  $\tilde{Q}\tilde{\Sigma}\tilde{R}^\top$  为

$$[\Sigma \mid Q^\top W u_{\ell+1} \mid Q^\top W u_{\ell+2} \mid \cdots \mid Q^\top W u_{\ell+s}] \in \mathbb{R}^{k \times (k+s)} \quad (4.14)$$

的标准薄 SVD。那么我们建议进行以下更新

$$Q \leftarrow Q\tilde{Q}, \quad \Sigma \leftarrow \tilde{\Sigma}, \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix} \tilde{R}. \quad (4.15)$$

此外, (4.15) 中的更新与算法 4 的输出完全相同。

注 2. 在实际计算中, 我们不计算  $Q\tilde{Q}$ , 只需要存储  $Q$  和  $\tilde{Q}$  以备将来更新。由于我们只需要计算  $\tilde{Q} r$  次, 那么它们的乘积不会失去正交性。因此, 我们对 Brand 在 [4] 中的问题的回答是: 对  $\tilde{Q}$  的重正交化是不需要的。此外, 为了更新右奇异子空间, 我们不计算矩阵的伪逆。值得提及的是, 在实践中, 我们不通过将  $\tilde{R}$  分割成  $\begin{bmatrix} \tilde{R}_1 \\ \tilde{R}_2 \end{bmatrix}$  来形成矩阵  $\begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix}$ , 然后  $R$  通过  $\begin{bmatrix} R\tilde{R}_1 \\ \tilde{R}_2 \end{bmatrix}$  更新。更新矩阵  $R$  的总计算成本为  $\mathcal{O}(nr^3)$ 。由于  $r \ll m, n$ , 该成本可以忽略。然而, 值得提及的是, [4] 中的技术可以通过计算  $r$  次小矩阵的伪逆来更新右奇异空间, 从而将计算成本降低到  $\mathcal{O}(nr^2)$ 。

一旦完成了  $U_{\ell+s}$  的 SVD, 我们接着更新  $U_{\ell+s+1}$  的 SVD。为了更好地描述算法。我们假设  $Q\Sigma R^\top$  是  $U_\ell$  的核心 SVD,  $\hat{Q}\hat{\Sigma}\hat{R}^\top$  是  $U_{\ell+s}$  的核心 SVD, 那么

$$\hat{Q} = Q\hat{Q}, \quad \hat{R} = \begin{bmatrix} R & 0 \\ 0 & I_s \end{bmatrix} \tilde{R}, \quad (4.16)$$

其中  $\hat{Q}$  和  $\tilde{R}$  分别是 (4.14) 的左、右奇异矩阵。



根据我们的假设，当将  $u_{\ell+s+1}$  投影到由  $Q$  的列张成的空间上时，其残差大于  $\text{tol}$ 。为了保持相同的精度，我们应该遵循 Brand 的想法来计算将  $u_{\ell+s+1}$  投影到由  $\hat{Q}$  的列张成的空间上的残差。

换句话说，对于那些列，我们必须计算两次残差。因此，这些重复的计算成本可能很高。接下来，我们表明可以避免这种成本。

为此，我们遵循 Brand 的想法，通过以下方式构造  $[U_{\ell+s} \mid u_{\ell+s+1}]$  的 SVD

- (1) 令  $e = u_{\ell+s+1} - \hat{Q}\hat{Q}^\top W u_{\ell+s+1}$ ，并令  $p = \|e\|_W$  和  $\tilde{e} = e/p$
- (2) 找到  $\bar{Y} = \begin{bmatrix} \hat{\Sigma} & \hat{Q}^\top u_{\ell+s+1} \\ 0 & p \end{bmatrix}$  的完全 SVD，并令  $\hat{Q}\Sigma\hat{R}^\top$  为  $\bar{Y}$  的 SVD，然后
- (3) 通过下式更新  $[U_{\ell+s} \mid u_{\ell+s+1}]$  的 SVD

$$[U_{\ell+s} \mid u_{\ell+s+1}] = \left( \left[ \hat{Q} \mid \tilde{e} \right] \hat{Q} \right) \Sigma \left( \begin{bmatrix} \hat{R} & 0 \\ 0 & 1 \end{bmatrix} \tilde{R} \right)^\top. \quad (4.17)$$

接下来，我们讨论上述三个步骤。首先，由于引理 2，步骤 (1) 中的计算是不必要的。其次，我们确实需要在步骤 (2) 中计算  $\hat{Q}^\top u_{\ell+s+1}$ 。然而，由于  $\hat{Q}^\top u_{\ell+s+1} = \hat{Q}^\top Q^\top u_{\ell+s+1}$ ，并且我们已经计算了  $Q^\top u_{\ell+s+1}$ ，那么更新  $\hat{Q}^\top Q^\top u_{\ell+s+1}$  几乎不花时间。对于步骤 (3)，通过使用 (4.16) 我们有

$$\begin{aligned} [U_{\ell+s} \mid u_{\ell+s+1}] &= \left( \left[ Q\hat{Q} \mid \tilde{e} \right] \hat{Q} \right) \Sigma \left( \begin{bmatrix} \hat{R} & 0 \\ 0 & 1 \end{bmatrix} \tilde{R} \right)^\top \\ &= \left( [Q \mid \tilde{e}] \begin{bmatrix} \hat{Q} & 0 \\ 0 & 1 \end{bmatrix} \hat{Q} \right) \Sigma \left( \begin{bmatrix} \hat{R} & 0 \\ 0 & 1 \end{bmatrix} \tilde{R} \right)^\top. \end{aligned} \quad (4.18)$$

这表明通过添加向量  $\tilde{e}$  来更新左奇异矩阵  $Q$ ，计算  $\begin{bmatrix} \hat{Q} & 0 \\ 0 & 1 \end{bmatrix} \hat{Q}$  然后存储它，计算成本为  $\mathcal{O}(r^2)$ 。对于右奇异矩阵，我们直接计算它，时间复杂度为  $\mathcal{O}(nr^2)$ 。步骤 (3) 将执行  $r$  次，那么这一步的总 CPU 时间为  $\mathcal{O}(nr^3)$ 。我们注意到，如果使用 [4] 中的技术，通过计算  $r$  次矩阵的伪逆，计算成本可以降低到  $\mathcal{O}(nr^2)$ 。

## 4.2 重正交化

正如我们在第 2 节中声称的，实践中的微小数值误差可能导致正交性损失。在 [7,13] 中对矩阵  $\left( \left[ \hat{Q} \mid \tilde{e} \right] \hat{Q} \right)$  进行重正交化是必要的，并且由于算

法 3 中的两个循环，计算成本很高。

观察恒等式 (4.18)，我们注意到正交矩阵  $\hat{Q}$  和  $\tilde{Q}$  很小，并且只有  $r$  个它们相乘，它们可以在合理的数值精度内保持正交性。因此，我们不需要对这些小的正交矩阵执行重正交化。这回答了 Brand 在 [4] 中提出的问题。

基于我们在例 2 中的数值实验。矩阵  $[Q | \tilde{e}]$  可能会非常快地失去正交性。因此，我们考虑对矩阵  $[Q | \tilde{e}]$  进行重正交化。我们注意到，这是必要的，并且 Brand 的原始算法中没有包括这一点。

根据  $[Q | \tilde{e}]$  的构造，我们可以递归地对其进行重正交化：仅对新添加的向量  $\tilde{e}$  应用  $W$ -加权 Gram-Schmidt，因为矩阵  $Q$  在先前的步骤中已经过重正交化，如果  $\tilde{e}$  与  $Q$  的第一列之间的加权内积大于某个容差。此外，如果  $\tilde{e} = 0$ ，那么最后一列将被截断。换句话说，我们不需要在截断部分 ( $\|p\|_W < \text{tol}$ ) 执行重正交化；更多细节请参见算法 7。

#### 算法 7 (增量SVD (III))

输入:  $Q \in \mathbb{R}^{m \times k}$ ,  $\Sigma \in \mathbb{R}^{k \times k}$ ,  $R \in \mathbb{R}^{k \times k}$ ,  $u_{+1} \in \mathbb{R}^m$ ,  $W \in \mathbb{R}^{m \times m}$ ,  $\text{tol}$ ,  $V$ ,  $Q_0$ ,  $q$

```

1: 设  $d = Q^T (W u_{+1})$ ;  $e = u_{+1} - Q d$ ;  $p = (e^T W e)^{1/2}$ 
2: if  $p < \text{tol}$  then
3:    $q = q + 1$ 
4:   设  $V\{q\} = Q_0^T d$ 
5: else
6:   if  $q > 0$  then
7:     设  $Y = [\Sigma, \text{cell2mat}(V)]$ 
8:      $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, \text{'econ'})$ 
9:     设  $Q_0 = Q_0 Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R_1 = R_Y(1:k, 1:\text{end}-1)$ ,  $R_2 = R_Y(k+1, 1:\text{end}-1)$ 
10:     $R = [R * R_1; R_2]$ ;
11:    设  $d = Q_Y^T d$ 
12:  end if
13:  设  $e = e / p$ 
14:  if  $|e^T W Q(:,1)| > \text{tol}$  then
15:     $e = e - Q (Q^T (W e))$ ;  $p_1 = (e^T W e)^{1/2}$ ;  $e = e / p_1$ 
16:  end if
17:  设  $Y = [\Sigma, d; 0, p]$ 
18:   $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y)$ 
19:  设  $Q_0 = [Q_0, 0; 0, 1] Q_Y$ ,  $Q = [Q, e]$ ,  $\Sigma = \Sigma_Y$ ,  $R = [R, 0; 0, 1] R_Y$ 

```

```

20:   设  $V = []$ ,  $q = 0$ 
21: end if
22: return  $Q, \Sigma, R, V, Q0, q$ 

```

### 4.3 算法 7 的复杂度

我们将自己限制在  $W = I$  的情况来讨论算法 7。回顾  $U \in \mathbb{R}^{m \times n}$  具有低秩  $r$ 。

1. 第 1 行: 这与算法 5 完全相同, 复杂度为  $\mathcal{O}(mnr)$ 。2. 第 3-4 行: 向量  $d \in \mathbb{R}^k$  且  $k \leq r$ , 因此这一步在 CPU 时间和内存存储方面几乎可以忽略。Brand 的方法没有这一步。3. 第 7-11 行: 主要成本是更新  $R$ , CPU 时间小于  $(nr^3)$ 。如前所述, 成本可以降低到  $\mathcal{O}(nr^2)$ 。我们将其留在此处, 因为  $r \ll m$ 。4. 第 14-16 行: 重正交化的 CPU 时间每次为  $\mathcal{O}(mr)$ , 这部分将执行  $\mathcal{O}(r)$  次, 因此总复杂度为  $\mathcal{O}(mr^2)$ 。这部分在算法 5 中没有包含, 但对于某些数据集是必要的。5. 第 17-20 行: 总复杂度小于  $\mathcal{O}(nr^3)$ 。同样, 这可以通过使用 [4] 中的技术降低到  $\mathcal{O}(nr^2)$ 。

根据我们的假设,  $r \ll m, n$ , 因此算法 7 的 CPU 时间为  $\mathcal{O}(mnr)$ 。

我们注意到, 算法 7 的输出  $V$  可能不为空。这意味着算法 7 的输出不是  $U$  的 SVD。因此, 我们必须更新  $V$  中向量的 SVD。我们在算法 8 中给出了这一步的实现, 复杂度为  $\mathcal{O}(mr^2)$ 。最后, 我们在算法 9 中完成了完整的实现。

算法 8 (增量 SVD (III) 最终检查)

```

输入:  $Q \in \mathbb{R}^{m \times k}, \Sigma \in \mathbb{R}^{k \times k}, R \in \mathbb{R}^{k \times k}, V, Q0, q$ 
1: if  $q > 0$  then
2:   设  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
3:    $[Q\_Y, \Sigma\_Y, R\_Y] = \text{svd}(Y, 'econ');$ 
4:   设  $Q = Q (Q0 Q\_Y), \Sigma = \Sigma\_Y, R1 = R\_Y(1:k, 1:\text{end}-1),$ 
5:    $R2 = R\_Y(k+1, 1:\text{end}-1), R = [R * R1; R2];$ 
6: else
7:   设  $Q = Q Q0$ ;
8: end if
9: return  $Q, \Sigma, R$ .

```

算法 9 (完全增量 SVD (III))

```

输入:  $W \in \mathbb{R}^{m \times m}$ , tol
1: 获取  $u_1$ ;
2:  $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$ ; % 算法 1
3: 设  $V = []$ ;  $Q_0 = I$ ;  $q = 0$ ;
4: for  $i = 2, \dots, n$  do
5:   获取  $u_i$ 
6:    $[q, V, Q_0, Q, \Sigma, R] = \text{UpdateISVD3}(q, V, Q_0, Q, \Sigma, R, u_i, W, \text{tol})$ ; % 算法 7
7: end for
8:  $[Q, \Sigma, R] = \text{UpdateISVD3check}(q, V, Q_0, Q, \Sigma, R)$ ; % 算法 8
9: return  $Q, \Sigma, R$ 

```

例 3. 在此示例中，我们重新审视例 1 以测试算法 9 的效率和准确性。我们在图 4 中展示了左奇异向量矩阵的正交性，其中

$$\mathcal{E}_W(Q) = \|I - Q^\top W Q\|, \quad \mathcal{E}_I(\tilde{Q}) = \|I - \tilde{Q}^\top \tilde{Q}\|.$$

我们看到我们的算法 9 保持了良好的正交性。

图 4: 例 3: 上图:  $W = I$  时  $\tilde{Q}$  和  $Q$  的正交性。下图:  $W = M$  时  $\tilde{Q}$  和  $Q$  的正交性。

接下来，我们在表 2 中展示了算法 7 主要部分的 CPU 时间（秒）。整个模拟对于  $W = I$  和  $W = M$  分别花费约 34 秒和 118 秒。与表 1 相比，CPU 时间大大减少，特别是在加权范数设置下 ( $W \neq I$ )。此外，投影部分（算法 7 第 1 行）占整个模拟时间的约 99

表 2: 例 3: 使用算法 9 时两种不同加权范数的 CPU 时间（秒）。

|         | 算法 7 第 1 行 | 第 3-4 行 | 第 7-11 行 | 第 14-16 行 | 第 17-20 行 |
|---------|------------|---------|----------|-----------|-----------|
| $W = I$ | 33.457     | 0.2416  | 0.0327   | 0.1251    | 0.2882    |
| $W = M$ | 116.41     | 0.1870  | 0.0650   | 0.6526    | 0.7120    |

## 5 另一种截断

对于许多数据集，它们可能有大量非零奇异值，但其中大多数非常小。如果不截断这些小奇异值，增量 SVD 算法可能会持续计算它们，因此计算

成本巨大。遵循 [7, 算法 4], 如果最后几个奇异值小于容差, 我们设置另一种截断。

接下来, 我们表明, 只需检查最后一个奇异值即可。令  $Q\Sigma R^\top$  为  $U_\ell$  的核心 SVD,  $e = u_{\ell+1} - QQ^\top W u_{\ell+1}$ , 且  $p = \|e\|_W > \text{tol}$ , 则

$$U_{\ell+1} = [Q \mid \tilde{e}] \underbrace{\begin{bmatrix} \Sigma & Q^\top W u_{\ell+1} \\ 0 & p \end{bmatrix}}_Y \underbrace{\begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}}_Y^\top = [Q \mid \tilde{e}] Q_Y \Sigma_Y R_Y^\top \underbrace{\begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix}}_Y^\top.$$

接下来, 我们估计  $Y$  的奇异值。

引理 3. 假设  $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_k)$  且  $\sigma_1 \geq \dots \geq \sigma_k$ , 以及  $\Sigma_Y = \text{diag}(\mu_1, \dots, \mu_{k+1})$  且  $\mu_1 \geq \dots \geq \mu_{k+1}$ 。那么我们有

$$\begin{aligned} \mu_{k+1} &\leq p, \\ \mu_{k+1} &\leq \sigma_k \leq \mu_k \leq \sigma_{k-1} \leq \dots \leq \sigma_1 \leq \mu_1. \end{aligned} \quad (5.1a)$$

证明。首先, 我们考虑  $Y^\top Y$

$$Y^\top Y = \begin{bmatrix} \Sigma^2 & \Sigma Q^\top W u_{\ell+1} \\ u_{\ell+1}^\top W Q \Sigma & p^2 + u_{\ell+1}^\top W Q Q^\top W u_{\ell+1} \end{bmatrix}.$$

通过使用 Cauchy 交错定理 [9, 定理 1] 我们有

$$\mu_{k+1} \leq \sigma_k \leq \mu_k \leq \sigma_{k-1} \leq \dots \leq \sigma_1 \leq \mu_1.$$

接下来, 我们取  $e = [0, 0, \dots, 1]$  得到

$$p^2 = e Y Y^\top e \geq \min_{\|x\|=1} x Y Y^\top x = \lambda_{\min}(Y Y^\top) = \mu_{k+1}^2.$$

不等式 (5.1a) 表明, 无论  $p$  有多大,  $Y$  的最后一个奇异值都可能非常小。也就是说, 我们为  $p$  设置的容差不能避免算法持续计算非常小的奇异值。因此, 如果数据有大量非常小的奇异值, 则需要进行另一种截断。不等式 (5.1b) 保证我们只有  $Y$  的最后一个奇异值可能小于容差。因此, 我们只需要检查最后一个。

(i) 如果  $\Sigma_Y(k+1, k+1) \geq \text{tol}$ , 则

$$Q \leftarrow [Q \mid \tilde{e}] Q_Y, \quad \Sigma \leftarrow \Sigma_Y, \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y. \quad (5.2)$$

(ii) 如果  $\Sigma_Y(k+1, k+1) < \text{tol}$ , 则

$$Q \leftarrow [Q \mid \tilde{e}]Q_Y(:, 1:k), \quad \Sigma \leftarrow \Sigma_Y(1:k, 1:k), \quad R \leftarrow \begin{bmatrix} R & 0 \\ 0 & 1 \end{bmatrix} R_Y(:, 1:k). \quad (5.3)$$

正如我们在第 4 节中讨论的, 我们可以通过仅更新小矩阵  $Q_Y$  并分别存储  $Q_Y$  和  $[Q \mid \tilde{e}]$  来避免 (5.2) 中的时间复杂度  $\mathcal{O}(mr^2)$ 。不幸的是, 为了截断非常小的奇异值, 似乎很难在  $\mathcal{O}(r^2)$  时间内找到两个正交矩阵  $\tilde{Q} \in \mathbb{R}^{m \times k}$  和  $\tilde{Q} \in \mathbb{R}^{k \times k}$  使得

$$[Q \mid \tilde{e}]Q_Y(:, 1:k) = \tilde{Q}\tilde{Q}.$$

因此, 我们直接使用 (5.2) 和 (5.3) 来更新矩阵  $Q$  和  $R$ 。假设数据有  $d$  个非零奇异值, 那么这种新的截断需要  $\mathcal{O}((m+n)r^2d)$  时间, 这里  $r$  是大于容差的奇异值的数量。

算法 10 (增量SVD (IV))

```

输入:  $Q \in \mathbb{R}^{m \times k}$ ,  $\Sigma \in \mathbb{R}^{k \times k}$ ,  $R \in \mathbb{R}^{k \times k}$ ,  $u_{+1} \in \mathbb{R}^m$ ,  $W \in \mathbb{R}^{m \times m}$ ,  $\text{tol}$ ,  $V$ ,  $Q_0$ ,  $q$ 
1: 设  $d = Q^T (W u_{+1})$ ;  $e = u_{+1} - Q d$ ;  $p = (e^T W e)^{1/2}$ ;
2: if  $p < \text{tol}$  then
3:    $q = q + 1$ ;
4:   设  $V\{q\} = d$ ;
5: else
6:   if  $q > 0$  then
7:     设  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
8:      $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, \text{'econ'})$ ;
9:     设  $Q_0 = Q_0 Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R_1 = R_Y(1:k, 1:\text{end}-1)$ ,
10:     $R_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = [R * R_1; R_2]$ ;
11:    设  $d = Q_Y^T d$ 
12:   end if
13:   设  $e = e / p$ ;
14:   if  $|e^T W Q(:, 1)| > \text{tol}$  then
15:      $e = e - Q (Q^T (W e))$ ;  $p_1 = (e^T W e)^{1/2}$ ;  $e = e / p_1$ ;
16:   end if
17:   设  $Y = [\Sigma, d; 0, p]$ ;

```

```

18:    [Q_Y,  $\Sigma_Y$ , R_Y] = svd(Y);
19:    if  $\Sigma_Y(k+1, k+1) > \text{tol}$  then % 注意此处应为  $\Sigma_Y$ 
20:        设  $Q_0 = [Q_0, 0; 0, 1]$   $Q_Y$ ,  $Q = [Q, e]$   $Q_0$ ,  $\Sigma = \Sigma_Y$ ,  $R_1 = R_Y(1:k, 1:\text{end}-1)$ ,
21:         $R_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = [R * R_1; R_2]$ ,  $Q_0 = I_{\{k+1\}}$ ;
22:    else
23:        设  $Q_0 = [Q_0, 0; 0, 1]$   $Q_Y$ ,  $Q = [Q, e]$   $Q_0(:, 1:k)$ ,
24:         $\Sigma = \Sigma_Y(1:k, 1:k)$ ,  $R = [R, 0; 0, 1]$   $R_Y(:, 1:k)$ ,  $Q_0 = I_k$ 
25:    end if
26:     $V = []$ ;  $q = 0$ ;
27: end if
28: return  $Q, \Sigma, R, V, Q_0, q$ 

```

接下来，我们在算法 11 和 12 中完成完整的实现。

算法 11 (增量SVD (IV) 最终检查)

输入:  $Q \in \mathbb{R}^{m \times k}$ ,  $\Sigma \in \mathbb{R}^{k \times k}$ ,  $R \in \mathbb{R}^{k \times k}$ ,  $V$ ,  $Q_0$ ,  $q$

```

1: if  $q > 0$  then
2:     设  $Y = [\Sigma \mid \text{cell2mat}(V)]$ ;
3:      $[Q_Y, \Sigma_Y, R_Y] = \text{svd}(Y, \text{'econ'})$ ;
4:     设  $Q = Q Q_Y$ ,  $\Sigma = \Sigma_Y$ ,  $R_1 = R_Y(1:k, 1:\text{end}-1)$ ,
5:      $R_2 = R_Y(k+1, 1:\text{end}-1)$ ,  $R = [R * R_1; R_2]$ ;
6: end if
7: return  $Q, \Sigma, R$ .

```

算法 12 (完全增量SVD (IV))

输入:  $W \in \mathbb{R}^{m \times m}$ ,  $\text{tol}$

```

1: 获取  $u_1$ .
2:  $[Q, \Sigma, R] = \text{InitializeISVD}(u_1, W)$  % 算法 1
3: 设  $V = []$ ;  $Q_0 = 1$ ;  $q = 0$ .
4: for  $i = 2, \dots, n$  do
5:     获取  $u_i$ 
6:      $[q, V, Q_0, Q, \Sigma, R] = \text{UpdateISVD4}(q, V, Q_0, Q, \Sigma, R, u_i, W, \text{tol})$  % 算法 10
7: end for
8:  $[Q, \Sigma, R] = \text{UpdateISVD4check}(q, V, Q_0, Q, \Sigma, R)$  % 算法 11
9: return  $Q, \Sigma, R$ 

```

## 6 结论

在本文中，我们回答了 Brand 在 [4] 中提出的开放问题。我们证明了新算法的输出与 Brand 原始算法的输出相同。数值实验表明，我们的新算法不仅节省了 CPU，而且为左奇异向量矩阵保持了非常好的正交性。未来，我们将把我们的新算法与 [1] 中的许多其他增量 SVD 算法进行比较。除此之外，我们将把这个新算法应用于许多不同的问题，例如模型降阶和分数阶 PDE。

## 致谢

作者感谢王立涵博士对引理 3 的证明。

## 参考文献

- [1] C. BACH, D. CEGLIA, L. SONG, AND F. DUDDECK, Randomized low-rank approximation methods for projection-based model order reduction of large nonlinear dynamical problems, *Internat. J. Numer. Methods Engrg.*, 118 (2019), pp. 209-241.
- [2] M. W. BERRY, Large-scale sparse singular value computations, *The International Journal of Supercomputing Applications*, 6 (1992), pp. 13-49.
- [3] M. BRAND, Incremental singular value decomposition of uncertain data with missing values, in *European Conference on Computer Vision*, Springer, 2002, pp. 707-720.
- [4] M. BRAND, Fast low-rank modifications of the thin singular value decomposition, *Linear Algebra Appl.*, 415 (2006), pp. 20-30.
- [5] H. FAREED AND J. R. SINGLER, A note on incremental POD algorithms for continuous time data, *Appl. Numer. Math.*, 144 (2019), pp. 223-233.



- [6] H. FAREED AND J. R. SINGLER, Error analysis of an incremental proper orthogonal decomposition algorithm for PDE simulation data, *J. Comput. Appl. Math.*, 368 (2020), pp. 1125-1135.
- [7] H. FAREED, J. R. SINGLER, Y. ZHANG, AND J. SHEN, Incremental proper orthogonal decomposition for PDE simulation data, *Comput. Math. Appl.*, 75 (2018), pp. 1942-1960.
- [8] M. GHASHAMI, E. LIBERTY, J. M. PHILLIPS, AND D. P. WOODRUFF, Frequent directions: simple and deterministic matrix sketching, *SIAM J. Comput.*, 45 (2016), pp. 1762-1792.
- [9] S.-G. HWANG, Cauchy's interlace theorem for eigenvalues of Hermitian matrices, *Amer. Math. Monthly*, 111 (2004), pp. 157-159.
- [10] M. A. IWEN AND B. W. ONG, A distributed and incremental SVD algorithm for agglomerative data analysis on large networks, *SIAM J. Matrix Anal. Appl.*, 37 (2016), pp. 1699-1718.
- [11] L. LIN AND Y. TONG, Low-rank representation of tensor network operators with long-range pairwise interactions, *SIAM J. Sci. Comput.*, 43 (2021), pp. A164-A192.
- [12] N. MASTRONARDI, E. E. TYRTYSHNIKOV, AND P. VAN DOOREN, A fast algorithm for updating and downsizing the dominant kernel principal components, *SIAM J. Matrix Anal. Appl.*, 31 (2010), pp. 2376-2399.
- [13] G. M. OXBERRY, T. KOSTOVA-VASSILEVSKA, W. ARRIGHI, AND K. CHAND, Limited-memory adaptive snapshot selection for proper orthogonal decomposition, *Internat. J. Numer. Methods Engrg.*, 109 (2017), pp. 198-217.